Hướng dẫn chi tiết: Triển khai CloudShop AI lên Cloudflare

Tài liệu này hướng dẫn từng bước quy trình triển khai (deploy) toàn bộ ứng dụng **CloudShop AI** (bao gồm Giao diện SvelteKit build tĩnh, Backend Hono.js Worker, Database Cloudflare D1 và Chatbot Workers AI) lên nền tảng Cloudflare Serverless.

Mục lục

1. Kiến trúc triển khai
2. Yêu cầu tiền đề (Prerequisites)
3. Bước 1: Đăng nhập & Xác thực Cloudflare
4. Bước 2: Khởi tạo Database Cloudflare D1 (Remote)
5. Bước 3: Cấu hình `wrangler.toml`
6. Bước 4: Nạp Schema & Dữ liệu mẫu vào D1 Production
7. Bước 5: Cài đặt Dependencies
8. Bước 6: Thực hiện Deploy lên Cloudflare
9. Bước 7: Kiểm thử & Nghiệm thu sau khi Deploy
10. Bước 8: Cấu hình Tên miền riêng (Custom Domain - Tùy chọn)
11. Vận hành & Cập nhật dữ liệu từ xa
12. Xử lý sự cố thường gặp (Troubleshooting)

---

1. Kiến trúc triển khai

Dự án triển khai theo mô hình **All-in-One Serverless** trên Cloudflare:

- **Giao diện (Frontend):** SvelteKit 5 + Tailwind CSS 4, mã nguồn trong `frontend/`, build tĩnh ra `public/` bằng `@sveltejs/adapter-static`, rồi được lưu trữ và phân phối qua **Cloudflare Workers Static Assets**.
- **Backend API:** Viết bằng **Hono.js (TypeScript)** (`src/index.ts`), chạy native trực tiếp trên runtime V8 của Cloudflare Workers với tốc độ phản hồi tức thì (< 5ms cold start).
- **Cơ sở dữ liệu (SQL): Cloudflare D1** (Serverless SQLite), phân tán toàn cầu với độ trễ thấp.
- **Cơ sở dữ liệu Vector (Vector DB): Cloudflare Vectorize** kết hợp model embedding đa ngôn ngữ `@cf/baai/bge-m3` để tìm kiếm tương đồng ngữ nghĩa (Semantic Vector Search).

- **Trí tuệ nhân tạo (AI): Cloudflare Workers AI** chạy mô hình ngôn ngữ lớn (LLM `@cf/google/gemma-4-26b-a4b-it`) trực tiếp trên hạ tầng GPU của Cloudflare theo quy trình chuẩn **RAG (Retrieval-Augmented Generation)**.
- **Bảo mật:** Không để lộ API Key ở frontend nhờ cơ chế **Cloudflare Bindings** (`env.DB`, `env.AI`, `env.VECTORIZE`, `env.ASSETS`).

```
None
Trình duyệt người dùng (Hỏi: "Có laptop nào cho sinh viên?")
       |
       ▼
 [ Cloudflare Edge Network ]
       ├── /api/chat ──► [ Hono.js Worker (src/index.ts) ]
       |                       ├── 1. Tạo Query Vector (BGE-M3) ──► [ Workers AI ]
       |                       ├── 2. Vector Search (Top 4)     ──► [ Cloudflare
Vectorize ]
       |                       ├── 3. Lấy đúng SP khớp          ──► [ Cloudflare D1
]
       |                       └── 4. Sinh câu trả lời RAG      ──► [ Workers AI
(Gemma 26B) ]
       |
       └── /* (Static) ──► [ Static Assets (public/) ]
```

---

2. Yêu cầu tiền đề (Prerequisites)

Trước khi thực hiện, đảm bảo máy tính đã cài đặt các công cụ sau:

1. **Tài khoản Cloudflare:** Đã đăng ký tại https://dash.cloudflare.com (Gói miễn phí - Free tier là đủ).
2. **Node.js & npm:** Node.js `>= 18.x` hoặc mới hơn.

```
Shell
node -v
npm -v
```

3. **Wrangler CLI:** Được cài đặt sẵn qua `package.json` hoặc chạy qua `npx wrangler`.
4. **Wrangler CLI:** Được tích hợp qua `npx wrangler`.

Bước 1: Đăng nhập & Xác thực Cloudflare

Mở Terminal tại thư mục gốc của project (`d:\cloudflare-shop-ai`):

Cách 1: Đăng nhập qua trình duyệt (Khuyến nghị cho cá nhân)

Chạy lệnh:

```shell
npx wrangler login
```

- Trình duyệt sẽ tự động bật lên trang đăng nhập Cloudflare.
- Nhấn **Allow** để cấp quyền cho Wrangler CLI.

Kiểm tra thông tin tài khoản sau khi đăng nhập:

```shell
npx wrangler whoami
```

Kết quả hiển thị tên tài khoản và `Account ID` là bạn đã đăng nhập thành công.

Cách 2: Dùng API Token (Dành cho CI/CD hoặc môi trường không mở được browser)

1. Truy cập Cloudflare API Tokens.
2. Tạo Token có các quyền:
   - `Account` - `Workers D1`: Edit
   - `Account` - `Workers AI`: Edit
   - `Account` - `Workers Scripts`: Edit
   - `User` - `User Details`: Read
3. Gán biến môi trường trong PowerShell:

```powershell
$env:CLOUDFLARE_API_TOKEN="<token_cua_ban>"
```

Bước 2: Khởi tạo Database Cloudflare D1 (Remote)

Tạo một database D1 mới trên Cloudflare (khuyến nghị chọn vị trí `apac` - Châu Á Thái Bình Dương):

```shell
npx wrangler d1 create cloudflare-shop-db --location=apac
```

Output sau khi tạo sẽ có dạng như sau:

```none
✅ Successfully created DB 'cloudflare-shop-db'!

[[d1_databases]]
binding = "DB"
database_name = "cloudflare-shop-db"
database_id = "xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx"
```

> [!IMPORTANT] Hãy sao chép chuỗi `database_id` được hiển thị để cập nhật vào bước tiếp theo.

Bước 2b: Khởi tạo Index Cloudflare Vectorize (Vector Database)

Tạo một Vector Index mới trên Cloudflare dùng cho Semantic Search (khớp với model `@cf/baai/bge-m3` với 1024 chiều vector, khoảng cách cosine):

```shell
npx wrangler vectorize create cloudflare-shop-index --dimensions=1024
--metric=cosine
```

Output sau khi tạo:

```none
✅ Successfully created index 'cloudflare-shop-index'
```

Bước 3: Cấu hình `wrangler.toml`

Mở file `wrangler.toml` tại thư mục gốc dự án và kiểm tra các thông số (đã cấu hình sẵn `DB`, `AI` và `VECTORIZE`):

```toml
name = "cloudflare-shop-ai"
main = "src/index.ts"
compatibility_date = "2026-10-05"
```

```
[assets]
directory = "./public"
binding = "ASSETS"
run_worker_first = ["/api/*"]

[ai]
binding = "AI"

[[d1_databases]]
binding = "DB"
database_name = "cloudflare-shop-db"
database_id = "<DÁN_DATABASE_ID_VỪA_TẠO_VÀO_ĐÂY>"

[[vectorize]]
binding = "VECTORIZE"
index_name = "cloudflare-shop-index"

[observability]
enabled = true
```

Bước 4: Nạp Schema & Dữ liệu mẫu vào D1 Production

Sau khi cấu hình ID, chạy lệnh sau để tạo các bảng `products`, `policies` và chèn dữ liệu mẫu lên database cloud:

```shell
npx wrangler d1 execute cloudflare-shop-db --remote --file=./schema.sql
```

Xác minh dữ liệu đã vào database thành công:

```shell
npx wrangler d1 execute cloudflare-shop-db --remote --command="SELECT id, name, price FROM products LIMIT 3;"
```

Nếu terminal in ra danh sách sản phẩm như `CloudPhone X1`, `CloudBook Air 14`,... nghĩa là database remote đã sẵn sàng.

Bước 5: Cài đặt Dependencies (Hono.js & TypeScript)

Đảm bảo các thư viện Node.js được cài đặt đầy đủ cho cả Worker và giao diện:

```shell
npm install
npm --prefix frontend install
```

Build giao diện SvelteKit vào thư mục `public/` (bắt buộc trước khi deploy):

```shell
npm run build:frontend
```

Kiểm tra kiểu TypeScript của Worker trước khi deploy (tùy chọn):

```shell
npm run build
```

Bước 6: Thực hiện Deploy lên Cloudflare

Chạy lệnh deploy:

```shell
npm run deploy
```

(Lệnh này tự động chạy `npm run build:frontend` rồi `wrangler deploy`. Nếu muốn deploy Worker mà không build lại giao diện, dùng `npx wrangler deploy`.)

Wrangler sẽ thực hiện các việc:

1. Đóng gói mã nguồn Hono.js TypeScript (`src/index.ts`).
2. Tải và triển khai toàn bộ thư mục frontend `public/` lên Cloudflare Static Assets.
3. Liên kết các binding `DB` (D1), `AI` (Workers AI), `VECTORIZE` và `ASSETS`.
4. Phát hành ứng dụng trên mạng lưới toàn cầu của Cloudflare.

Khi hoàn tất, terminal sẽ in ra URL công khai:

```none
Total Upload: ... KiB / gzip: ... KiB
Uploaded cloudflare-shop-ai (x.xx sec)
Deployed cloudflare-shop-ai triggers
```

```
  https://cloudflare-shop-ai.<your-subdomain>.workers.dev
Current Version ID: xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx
```

Bước 6b: Đồng bộ dữ liệu vào Cloudflare Vectorize (Tạo Vector Embeddings)

Sau khi deploy thành công lần đầu, bạn cần kích hoạt job đồng bộ để tạo vector embedding cho toàn bộ sản phẩm và chính sách từ D1 lưu vào Cloudflare Vectorize:

```powershell
# Trên PowerShell:
Invoke-RestMethod -Uri
"https://cloudflare-shop-ai.<your-subdomain>.workers.dev/api/sync-vectors" -Method POST
```

Hoặc dùng `curl`:

```
Shell
curl -X POST
https://cloudflare-shop-ai.<your-subdomain>.workers.dev/api/sync-vectors
```

Kết quả trả về dạng:

```
JSON
{
  "ok": true,
  "message": "Đã đồng bộ dữ liệu vào Cloudflare Vectorize thành công!",
  "total_indexed": 11,
  "products_count": 6,
  "policies_count": 5,
  "embedding_model": "@cf/baai/bge-m3"
}
```

Bước 7: Kiểm thử & Nghiệm thu sau khi Deploy

Mở đường dẫn `https://cloudflare-shop-ai.<your-subdomain>.workers.dev` trên trình duyệt:

1. **Giao diện & Sản phẩm:**
   - Kiểm tra trang chủ hiển thị đầy đủ danh sách sản phẩm, giá bán, nhãn khuyến mãi.
   - Thử lọc danh mục: "Điện thoại", "Laptop", "Âm thanh",...
   - Thử thêm sản phẩm vào giỏ hàng, mở Drawer giỏ hàng và kiểm tra tính năng tăng/giảm số lượng.
2. **Kiểm tra API Backend:**
   - Truy cập `https://cloudflare-shop-ai.<your-subdomain>.workers.dev/api/health` -> Phải nhận về JSON:

```JSON
{"ok": true, "service": "cloudflare-shop-ai", "runtime": "Hono.js on Cloudflare Edge", "vectorize_enabled": true}
```

   - Truy cập `https://cloudflare-shop-ai.<your-subdomain>.workers.dev/api/products` -> Phải trả về mảng JSON các sản phẩm.
3. **Kiểm tra Chatbot AI (Vectorize RAG):**
   - Bấm vào nút chat nổi tròn màu xanh ở góc phải màn hình (hoặc nút "Hỏi AI tư vấn").
   - Thử các câu hỏi thực tế:
     - *"Shop có laptop nào dưới 20 triệu không?"*
     - *"Chính sách bảo hành và đổi trả của shop như thế nào?"*
     - *"So sánh tai nghe CloudBuds Pro với đồng hồ CloudWatch S"*
   - AI sẽ tự động tạo embedding câu hỏi -> tìm kiếm trong Vectorize -> trích xuất đúng sản phẩm liên quan từ D1 -> sinh câu trả lời kèm nhãn ⚡ **`Cloudflare Vectorize RAG`**!

Bước 8: Cấu hình Tên miền riêng (Custom Domain - Tùy chọn)

Nếu bạn muốn trang web chạy trên tên miền cá nhân (ví dụ: `shop.yourdomain.com`):

1. Đăng nhập vào Cloudflare Dashboard.
2. Vào mục **Workers & Pages** -> Chọn Worker **`cloudflare-shop-ai`**.
3. Chọn tab **Settings** -> **Domains & Routes**.

4. Bấm **Add** -> chọn **Custom Domain**.
5. Nhập tên miền (ví dụ: `shop.yourdomain.com`) và xác nhận.
6. Cloudflare sẽ tự động tạo bản ghi DNS và cấp chứng chỉ SSL HTTPS miễn phí.

Vận hành & Cập nhật dữ liệu từ xa

1. Xem nhật ký (Logs) trực tiếp theo thời gian thực

Để theo dõi request và debug lỗi khi có người truy cập:

```
Shell
npx wrangler tail
```

2. Thêm hoặc cập nhật sản phẩm mà không cần deploy lại code

Do dữ liệu lưu tại Cloudflare D1, bạn có thể thêm sản phẩm trực tiếp từ dòng lệnh:

```
Shell
npx wrangler d1 execute cloudflare-shop-db --remote --command="INSERT INTO
products (name, category, price, old_price, stock, short_description, description,
specs, badge, icon) VALUES ('CloudMouse Pro', 'Phụ kiện', 850000, 990000, 40,
'Chuột không dây công thái học', 'Hỗ trợ Bluetooth và 2.4G, pin sạc dùng 3 tháng',
'DPI 4000; Pin sạc; Bluetooth 5.3', 'Mới', '🖱️');"
```

Ngay sau khi thêm, F5 lại website và hỏi Chatbot, thông tin sản phẩm mới sẽ hiển thị lập tức!

3. Cập nhật chính sách shop

```
Shell
npx wrangler d1 execute cloudflare-shop-db --remote --command="UPDATE policies SET
content = 'Đổi trả miễn phí trong 15 ngày đầu nếu sản phẩm có lỗi từ nhà sản
xuất.' WHERE slug = 'returns';"
```

Xử lý sự cố thường gặp (Troubleshooting)

1. Lỗi `In a non-interactive environment, it's necessary to set a CLOUDFLARE_API_TOKEN`

- **Nguyên nhân:** Wrangler chưa được cấp quyền đăng nhập tài khoản Cloudflare.

- **Cách sửa:** Chạy lệnh `npx wrangler login` và xác nhận trên trình duyệt.

2. Lỗi `no such table: products` hoặc `/api/products` trả về lỗi 500

- **Nguyên nhân:** Database D1 trên Cloudflare chưa được khởi tạo bảng từ file `schema.sql`.
- **Cách sửa:** Chạy lệnh:

```shell
npx wrangler d1 execute cloudflare-shop-db --remote --file=./schema.sql
```

3. Chatbot phản hồi: *"Không thể trả lời lúc này"*

- **Nguyên nhân:**
  - Kiểm tra xem binding `[ai]` trong `wrangler.toml` đã được kích hoạt chưa.
  - Model `@cf/google/gemma-4-26b-a4b-it` có thể gặp tải cao tạm thời. Bạn có thể đổi sang model tương đương như `@cf/meta/llama-3.1-8b-instruct` bằng hằng số `CHAT_MODEL` trong `src/index.ts` nếu cần.
  - Chạy `npx wrangler tail` để xem chi tiết thông báo lỗi trả về từ Worker.

4. Giao diện frontend không cập nhật sau khi sửa code

- **Nguyên nhân:** Thư mục `public/` là **output của build**, không phải mã nguồn. Sửa trực tiếp file trong `public/` sẽ bị ghi đè ở lần build kế tiếp; ngược lại, sửa trong `frontend/` mà không build lại thì `public/` vẫn giữ bản cũ.
- **Cách sửa:** Sửa giao diện trong `frontend/src/`, chạy `npm run build:frontend`, rồi `npx wrangler deploy`. Mở tab ẩn danh hoặc nhấn `Ctrl + F5` để xóa cache trình duyệt.

5. Build giao diện báo lỗi resolve `$app/tsconfig`

- **Nguyên nhân:** SvelteKit chưa sinh thư mục `.svelte-kit/`.
- **Cách sửa:** Chạy `npm --prefix frontend run build`; nếu vẫn lỗi, đảm bảo `frontend/tsconfig.json` có `"extends": "./.svelte-kit/tsconfig.json"`.